CSCE 685: Directed Studies

Final Project Report

Offline LLM Optimization and Deployment on Android Architectures

---

Student: Agastya Todi (UIN: 835009977)

Supervisor: Professor Duncan Walker

Credits: 2.0 Credit Hours

Semester: Spring 2026

Date: April 28, 2026

GitHub Repository:
https://github.com/Agastya910/offline-android-llm

Texas A&M University
Department of Computer Science and Engineering

Abstract

This report documents the research and implementation work conducted for CSCE 685 (Directed Studies, Spring 2026) on the topic of deploying and optimizing large language models (LLMs) entirely offline on Android mobile hardware. The project used the MLC-LLM framework,built on top of Apache TVM,to compile, quantize, and execute Microsoft Phi-1.5 (1.3 billion parameters) natively on a mid-range Android device featuring a Qualcomm Snapdragon SoC. Model weights were converted to INT4 group quantization using the `q4f16_1` scheme, and the compiled library was linked into an Android application via a Kotlin–JNI–C++ bridge. The deployed system achieved a measured decode throughput of approximately **3–4 tokens per second** on the target device, with a time-to-first-token of approximately 800–1200 ms. This report is written so that an independent developer could re-implement the full pipeline from scratch using only this document and the accompanying repository.

Contents

1. Introduction and Motivation

The rapid expansion of large language models has been almost entirely predicated on access to massive cloud GPU infrastructure. Services like ChatGPT, Claude, and Gemini process every user query through data-center servers, introducing non-trivial latency, privacy exposure, and operational costs. For an increasing class of applications,personal health assistants, offline note-taking tools, real-time language translation in low-connectivity environments,this cloud dependency is a fundamental blocker.

Modern mobile SoCs (System-on-Chip) have advanced dramatically. Qualcomm Snapdragon 8-series processors now integrate Adreno GPUs capable of several TFLOPS of FP16 throughput, and dedicated NPUs (Neural Processing Units). Yet the software infrastructure required to harness these resources for LLM inference,model compression, compiler-level kernel fusion, memory layout optimization,has until recently required expert knowledge and bespoke toolchains.

This project aimed to close that gap: to take a capable, small-parameter language model (Phi-1.5, 1.3B parameters) and produce a fully offline Android application that runs it using only the device's own GPU, with zero external API calls. The primary research and engineering questions were:

1. Can INT4 (`q4f16_1`) quantization produce acceptable output quality for Phi-1.5 on Android?
2. What is the practical decode throughput (tokens/second) achievable on a mid-range Snapdragon device?
3. How does one build the complete MLC-LLM → Android NDK → Kotlin UI pipeline end to end?
4. What are the key failure modes and engineering challenges in this stack?

2. Background and Related Work

2.1. Phi-1.5 Language Model

Phi-1.5 is a 1.3-billion-parameter Transformer language model released by Microsoft Research in September 2023 [1]. Unlike most models of its generation which were trained on massive web-crawl corpora, Phi-1.5 was trained on a curated dataset of 30 billion "textbook-quality" synthetic tokens. Despite its small size, Phi-1.5 achieves near state-of-the-art performance on commonsense reasoning, language understanding, and coding benchmarks among models under 10 billion parameters [1].

Key architectural details:

- Architecture: Transformer with next-word prediction objective (causal LM)
- Parameters: 1.3 billion; Hidden size: 2048; 24 layers; 32 heads
- Training data: 30 billion tokens (synthetic, curated)
- Native precision: FP16; deployed at INT4 (`q4f16_1`)
- On-disk size: 2.84 GB (FP16), ≈730 MB (INT4)

2.2. Apache TVM Compiler

Apache TVM is an open-source machine learning compiler framework that targets CPUs, GPUs, and specialized accelerators [2]. It uses a tiered intermediate representation (IR) to separate hardware-independent graph-level optimizations (operator fusion, memory planning, layout transformations) from hardware-specific code generation (LLVM for CPU, SPIR-V for Vulkan, OpenCL C for Qualcomm Adreno).

The TVM Unity release introduced *Relax IR*, a unified compute/graph IR that enables end-to-end optimization of dynamic-shape workloads such as autoregressive LLM inference,a critical capability for this project.

2.3. MLC-LLM Framework

MLC-LLM (Machine Learning Compilation for LLMs) is built on top of TVM Unity and provides a complete pipeline for deploying any HuggingFace model to diverse hardware backends [3]. It handles weight format conversion and quantization, model library compilation to Android/arm64 shared objects, KV-cache management, dynamic shape handling, and a C-API for native Android integration. Most modern Snapdragon devices support Vulkan 1.1, which is preferred over OpenCL for lower driver overhead [4].

2.4. Prior Art on On-Device LLM Performance

Community benchmarks report that a Snapdragon Gen 2 device achieves roughly 9 tok/s on a 3B Phi-2 model via MLC Chat [5]. The MobileAIBench framework provides systematic cross-device evaluation of LLMs across quantization levels and model sizes [6]. The expected operating range for Phi-1.5 at INT4 on a mid-range device is 3–8 tok/s depending on the SoC generation and memory bandwidth.

3. System Architecture

The full software stack from HuggingFace model checkpoint to rendered tokens in the Android UI comprises four layers:

1. **Model Preparation (Host, Python):** Download FP16 weights from HuggingFace Hub; apply INT4 group quantization via `mlc_llm convert_weight`; compile to an Android arm64-v8a native library (`.tar` → `.so`) via `mlc_llm compile --device android`.
2. **Native Inference Engine (Android C++):** Pre-built `libmlc_llm.so` and `libtvm_runtime.so` (from the MLC-LLM Android SDK) provide the TVM runtime, Vulkan/OpenCL kernel dispatcher, and the streaming chat C API.
3. **JNI Bridge (Android C++ + Kotlin):** `mlc_jni.cpp` wraps the MLC C-API and exposes it via JNI. `MLCBridge.kt` declares `external fun` signatures. Streaming tokens are delivered via a native callback that invokes the Kotlin lambda through `JNIEnv::CallObjectMethod`.

4. **Compose UI (Kotlin):** `ChatScreen.kt` uses Jetpack Compose with a `MutableStateFlow<String>` for real-time token streaming. A performance overlay shows TTFT, TPS, and token count.

4. Technical Methodology

4.1. Quantization: `q4f16_1`

INT4 weight quantization reduces each FP16 weight to a 4-bit integer, reducing memory footprint by approximately 4×. The `q4f16_1` scheme uses *group quantization* with group size $g = 32$: each group stores an FP16 scale s and zero-point z. During inference, weights are dequantized on-the-fly:

$$\hat{w}_i = s \cdot (q_i - z), \quad q_i \in \{0, 1, \ldots, 15\}$$

Activations remain in FP16, so matrix multiplications operate in FP16 precision.

The approximate memory saving is:

$$\frac{\text{Size}_{q4f16\_1}}{\text{Size}_{fp16}} \approx \frac{4}{16} + \frac{2 \times 2}{32 \times 16} \approx 0.257$$

yielding ≈ 730 MB for Phi-1.5.

4.2. Kernel Compilation for Adreno GPU

For Qualcomm Adreno GPUs, MLC-LLM supports two compute backends:

- **Vulkan** (preferred): lower driver overhead, better memory bandwidth utilization on Adreno 730+.
- **OpenCL** (fallback): broader device compatibility for older SoCs.

The `--device android` compilation flag bundles kernels for both backends; the TVM runtime selects the appropriate one at application launch.

4.3. KV-Cache Memory Management

For Phi-1.5 with a 2048-token context window, 24 layers, 32 heads, and 64 head-dimension, the KV cache at FP16 requires approximately:

$$2 \times 2048 \times 24 \times 32 \times 64 \times 2\,\text{B} \approx 768\,\text{MB}$$

On 8 GB Android devices, combined model weight + KV cache + OS overhead can cause OOM errors. Mitigations applied:

- `context_window_size = 2048` (configurable in `mlc-chat-config.json`)
- `prefill_chunk_size = 512` (reduces peak activation memory)
- `resetChat()` called via JNI between conversations to flush KV cache

4.4. ONNX Export for Cross-Validation

As a secondary validation step, the conversion script supports exporting Phi-1.5 to ONNX format (`torch.onnx.export`, opset 17). This enables comparison of FP16 reference outputs vs. MLC INT4 outputs on identical prompts, providing a proxy measure of quantization-induced accuracy loss without a full perplexity evaluation.

5. Implementation Details

5.1. Repository Structure

```
csce685_offline_llm/
          scripts/
                01_setup_environment.sh  # venv + dependency installation
                02_convert_model.py      # HF download     MLC conversion
       Android compile
          benchmark/
                benchmark_host.py        # Host-side TPS/TTFT benchmarking
          android_app/
                app/src/main/
                    kotlin/.../
                          MainActivity.kt  # App lifecycle, coroutine
   launch
                          MLCBridge.kt     # JNI external function
   declarations
                          ChatScreen.kt    # Jetpack Compose streaming UI
                    cpp/
                        mlc_jni.cpp      # C++ JNI implementation
                        CMakeLists.txt   # NDK build config
          requirements.txt
          README.md
```

Listing 1: Repository layout

5.2. Model Conversion Command

```
# Step 1: Convert weights to INT4
python -m mlc_llm convert_weight \
    ./hf_weights/phi-1_5 \
    --quantization q4f16_1 \
    --output ./dist/phi-1_5-q4f16_1-MLC

# Step 2: Generate MLC config
python -m mlc_llm gen_config \
    ./hf_weights/phi-1_5 \
    --quantization q4f16_1 \
    --prefill-chunk-size 512 \
    --context-window-size 2048 \
    --output ./dist/phi-1_5-q4f16_1-MLC

# Step 3: Compile for Android arm64
python -m mlc_llm compile \
```

```
    ./dist/phi-1_5-q4f16_1-MLC \
    --device android \
    --output ./dist/phi-1_5-q4f16_1-android.tar
```

Listing 2: INT4 model conversion and Android compilation

5.3. Streaming Inference Flow

The streaming chat path proceeds as follows:

1. User taps Send in `ChatScreen.kt`.
2. `MainActivity.sendMessage()` launches a coroutine on `Dispatchers.IO` to avoid blocking the main thread.
3. `MLCBridge.chat()` calls the native `chatNative()` JNI function.
4. C++ `MLCEngineChatStream()` runs the prefill pass (full prompt encoding), then iteratively decodes tokens.
5. For each decoded token, the native callback fires, invoking the Kotlin `onToken` lambda via JNI, which appends to `_streamingOutput: MutableStateFlow<String>`.
6. Compose observes the flow and recomposes the text component, rendering each new token with <20 ms UI latency.
7. On generation completion, TTFT and TPS are computed and stored in `_perfMetrics: MutableStateFlow<PerfMetrics>`.

6. Results and Performance Analysis

6.1. Measured Performance

Table 1 summarizes the on-device performance measurements obtained running Phi-1.5 (`q4f16_1`) on a mid-range Android device with a Snapdragon 7xx SoC and 8 GB RAM.

Table 1: On-Device Inference Performance , Phi-1.5 INT4 (`q4f16_1`)

Metric	Measured Value	Notes
Decode Throughput (TPS)	3–4 tok/s	Primary result
Time to First Token (TTFT)	800–1200 ms	Prefill phase
Model size on-disk (INT4)	≈730 MB	Down from 2.84 GB FP16
Peak RAM during inference	≈1.5–2.0 GB	Incl. KV cache
Context window	2048 tokens	Config cap
Backend selected	Vulkan	Auto-selected at runtime

6.2. Comparison with Related Systems

The 3–4 tok/s decode throughput for Phi-1.5 at INT4 is consistent with community benchmarks: Phi-2 (3B) achieves ∼3 tok/s on a Snapdragon Gen 2 device via MLC Chat [5], and Mistral 7B Q4 achieves ∼4 tok/s on Snapdragon 8 Gen 3 [5]. The primary bottleneck is

memory bandwidth: at 4 tok/s the system loads ≈730 MB of weights per second, approaching the theoretical peak of mid-range mobile GPUs (∼50–70 GB/s at 4-bit access granularity).

6.3. Engineering Challenges

OOM on first load. The combined weight + KV cache + OS memory exceeded device limits. Resolved by capping `prefill_chunk_size = 512` and `context_window_size = 2048` in the MLC config.

JNI local reference overflow. The streaming callback fires from a native thread. Failure to call `env->DeleteLocalRef` on each token's `jstring` caused a JNI reference table overflow at approximately 200 tokens. Fixed by adding `DeleteLocalRef` immediately after each callback.

Vulkan driver inconsistencies. Several device/driver combinations required fallback to OpenCL for stable inference, underscoring the importance of building both backends into the compiled library.

NDK version pinning. Linking against pre-built MLC shared libraries requires NDK r26b exactly; r25 and r27 produced ABI mismatch linker errors against `libtvm_runtime.so`.

7. Future Work

3-bit quantization (`q3f16_1`). Expected to reduce model size to ∼550 MB and increase decode throughput by 20–30% due to reduced memory bandwidth consumption per token, at the cost of some perplexity increase.

Speculative decoding. A 125M-parameter draft model could propose multi-token continuations verified by Phi-1.5, potentially multiplying effective TPS by 2–4× with negligible quality loss [7].

Multi-device benchmarking. Systematic evaluation across flagship (Snapdragon 8 Gen 3), mid-range (Snapdragon 7xx), and budget (Snapdragon 6xx) devices to characterize the TPS–RAM–SoC relationship.

Perplexity evaluation. A formal WikiText-2 perplexity comparison between FP16, INT4, and INT3 variants is needed to rigorously quantify accuracy trade-offs.

Thermal and battery profiling. 10-minute sustained inference sessions should be profiled for thermal throttling (CPU/GPU frequency scaling) and battery drain rate.

8. Conclusions

This project successfully demonstrated end-to-end offline LLM inference on a consumer Android device using the MLC-LLM and Apache TVM open-source stack. Starting from the HuggingFace `microsoft/phi-1_5` checkpoint, a complete pipeline was built: INT4 weight quantization, TVM compilation to Vulkan/OpenCL kernels for arm64-v8a, a streaming Kotlin–JNI–C++ bridge, and a Jetpack Compose chat UI.

The achieved decode throughput of 3–4 tok/s is sufficient for interactive use and establishes a concrete baseline for optimization work. The engineering challenges encountered,JNI thread safety, OOM management, NDK version pinning, and Vulkan driver variance,are well-documented in the codebase and this report for future developers.

The broader implication is that privacy-preserving, offline LLM inference on everyday mobile hardware is now practically achievable with open-source tooling, requiring no cloud connectivity and no exposure of user data to external servers.

References

[1] Y. Li, S. Bubeck, R. Eldan et al., "Textbooks Are All You Need II: phi-1.5 technical report," *arXiv:2309.05463*, 2023. https://arxiv.org/abs/2309.05463

[2] T. Chen, T. Moreau, Z. Jiang et al., "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," *USENIX OSDI*, 2018.

[3] MLC-LLM Team, "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," *GitHub*, 2023. https://github.com/mlc-ai/mlc-llm

[4] MLC-LLM Documentation, "Android SDK , mlc-llm 0.1.0," https://llm.mlc.ai/docs/deploy/android.html

[5] r/LocalLLaMA, "Running LLM on Android (Snapdragon 8 Gen 3)," March 2024. https://www.reddit.com/r/LocalLLaMA/comments/1bpw9c7/

[6] MobileAIBench, "Benchmarking LLMs and LMMs for On-Device Use Cases," *arXiv*, 2025. https://openreview.net/forum?id=EEbRrNsiiD

[7] V. Chandra, "On-Device LLMs: State of the Union, 2026," https://v-chandra.github.io/on-device-llms/, January 2026.